

KubeCon EU 2027 — Talk 1: AI-Assisted Observability

Contents

Talk 1 — Full package: “The Agent Said ‘Successfully Deleted.’ Nothing Was Deleted.”	1
1. Title (Title Case, inclusive-language checked)	1
2. Description (problem → contribution → relevance, per the CFP’s own framing)	1
3. Answers to the CFP’s “Preparing to Submit” questions	2
4. Detailed 25-minute talk outline	2
5. Notes for reviewers / “is this a case study” field	3
6. Relevant CNCF / ecosystem projects	3
7. Speaker bio (draft)	4
8. One-line pitch (social/preview card)	4
9. Illustrative diagrams (slide candidates — not for the CFP form itself)	4
10. Before you submit — checklist	4

Talk 1 — Full package: “The Agent Said ‘Successfully Deleted.’ Nothing Was Deleted.”

Source: Chapter 28 of the book, “*AI-assisted observability: an agent that reads telemetry*” — grounded in real incidents from docs/incidents/ (the fourth replay maps directly onto 2026-07-28-ecs-alert-coverage-drift-silent-gap.md; the third replay maps onto 2026-07-13-cal-prod-eb-health-4xx-403-flood-false-page.md and 2026-06-22-cal-prod-hour1

Target track: Observability Day (primary) — also fits Agentics Day: MCP + Agents. **Format:** Solo, 25 min. **CFP deadline:** Oct 18, 2026, 23:59 CET. Notifications Dec 14, 2026. Event Mar 15, 2027, Barcelona. Submit at <https://sessionize.com/cncf-hosted-co-located-events-europe-2027/>

1. Title (Title Case, inclusive-language checked)

Primary: *The Agent Said “Successfully Deleted.” Nothing Was Deleted.* **Subtitle (if the form has one):** *What Four Real Incidents Taught Us About Trusting an AI Agent With Your Telemetry*

Alternates: - *Replaying Incidents on an AI Agent: Where It’s Right, and Where It Confidently Lies* - *The Missing Context Layer: What an AI Agent Doesn’t Know About Your Telemetry*

No gendered, ableist, or violent language in any option — clean against the LF Inclusive Language Initiative.

2. Description (problem → contribution → relevance, per the CFP’s own framing)

Every major telemetry vendor now ships an MCP server, and the pitch is identical everywhere: give an AI agent read access to your metrics, logs, and traces, and it will speed up alert triage. **We didn’t take that promise on faith.**

We replayed four real, already-resolved incidents from our own production history through an agent using the same MCP-based query tools an on-call engineer has, and compared its reasoning to the answer we already knew. Two replays went well — the agent independently reconstructed the correct root cause through a chain of evidence a human would recognize.

One is the one worth your time: without extra context, the agent read a derived health metric at face value and would have confidently escalated a false outage, because nothing in generic observability knowledge told it that metric was known to lie. Testing the agent’s write boundary surfaced a sharper problem still: a blocked write and a successful one return an identical “success” message, so the agent will honestly report an action that never happened.

This talk covers what we built in response — a small, on-demand “context layer” of system-specific pitfalls, and enforcing read-only access at the token’s permission scope rather than trusting the agent’s self-report — and gives the audience a concrete method (replay against your own resolved incidents) to test the same promise against their own stack, before they trust it in production.

(≈235 words — trim to fit whatever the live Sessionize field allows.)

3. Answers to the CFP’s “Preparing to Submit” questions

These three are meant to sharpen the pitch above, and some CFP forms ask for them directly as separate fields. Drafted from what you’ve told me about your goals — edit freely, especially #1, since only you know what you actually want out of this.

Q1 — What are you hoping to get from your presentation? Feedback from other teams already running MCP-based agents against production telemetry — whether they’ve hit the same false-success and wrong-store failure modes, and how they’ve drawn the human-approval line. A first public test of this material ahead of finishing the book’s chapter on it. Conversations with people building or operating MCP servers for telemetry platforms, since the read-only-enforcement finding is directly actionable for them.

Q2 — What do you expect the audience to gain from your presentation? A tested, not theoretical, answer to “should I trust an agent with my telemetry” — with the specific boundary (read-only at token scope, not at prompt) audience members can go implement the same week. A concrete replay methodology they can run against their *own* resolved incidents rather than taking any vendor’s triage claim on faith. Two named failure patterns (false success on a blocked write; querying the structurally wrong data store) they’ll now recognize and test for before they see them in production.

Q3 — How will your presentation help better the ecosystem? MCP standardized how agents connect to telemetry, and adoption is moving faster than operational guidance for it. This talk turns “read-only by default” from a vendor’s marketing bullet into a specific, verified architectural requirement (enforce at token scope, because a blocked and a successful write are indistinguishable in the agent’s own report) — a concrete pattern the ecosystem’s growing number of MCP-telemetry integrations can adopt directly, backed by a real production test rather than a demo.

4. Detailed 25-minute talk outline

(20–22 min content + 3–5 min Q&A is the common split for a 25-min solo slot — confirm against the actual room format once accepted.)

[0:00–1:30] Hook. Open on the title quote and the diagram: a blocked write and a real write returning an identical “success” message. One sentence on what “read-only” actually means here. Do not explain yet — let it sit as a mystery the talk resolves.

[1:30–3:00] The setup. MCP standardized agent-to-tool access; every major telemetry vendor ships a server; consistent industry pattern of “read-only by default, explicit flag for writes.” State the question plainly: does the “faster triage” promise survive contact with real incidents?

[3:00–4:00] The method. Introduce “replay”: take incidents you’ve *already solved*, give the agent the same MCP query tools a human had, let it work independently, and compare its reasoning to the known answer. Cheap, precise, and — crucially — reusable by anyone in the room on their own incident history.

[4:00–7:00] Replays 1 & 2 — where it worked. Fast-paced. Replay 1: an idle-timeout latency signature the agent correctly distinguished from a crash pattern, then redirected from “where was time spent” to “where was time lost.” Replay 2: a two-environment differential that ruled out the obvious-but-wrong suspect (rising DB load) by comparing against historical

baselines — a diagnosis that took a human team a day to get right the first time. Point: generic reasoning goes further than expected.

[7:00–11:00] Replay 3 — the trap. (Emotional core of the talk.) Show the eb-health-vs-5xx dashboard. The agent, without context, would have confirmed a “serious outage” a derived health metric claimed — while the authoritative load-balancer signal showed near-zero real errors. Real cause: an autoscaling misconfiguration in an hourly loop, not an outage at all. Land the line: *“the damage wouldn’t be that the agent did nothing — it would be that it did the wrong thing, fast, and with a convincing explanation.”*

[11:00–13:00] Replay 4 — the class of absence. Six stacked lapses discovered by accident, not by alert: dead alert links, telemetry mislabeled under a sibling’s name, suppressed failures. The finding that matters: an agent checks *observed* reality; an automated, code-written check of *declared* configuration is a separate, still-necessary thing. One doesn’t replace the other.

[13:00–14:00] The unifying idea: the context layer. A small, purpose-built document of system-specific pitfalls, loaded exactly when the agent is about to query — not preloaded into every session. Say plainly why on-demand beats permanent: an always-present giant file gets ignored; a loaded-on-cue one gets read.

[14:00–17:00] The sharper problem: false success. The read-only enforcement story. A blocked delete and a real delete return the same message — the agent has no way to internally tell them apart. Land the architectural point hard: *this boundary cannot live in a prompt or in trust of the agent’s report — it has to live in the access token’s permission scope*, because that’s the only place the real difference exists.

[17:00–19:00] The wrong-store trap. Two concrete, generalizable failure shapes: an error at one layer (edge load balancer) that structurally cannot appear in another store (backend traces) — a query against the wrong store returns a convincing empty zero, not an error. And a percentile computed over a handful of requests, which is a sampling-size artifact, not a measurement.

[19:00–21:00] It’s not just us. Cite the external confirmation already in the book chapter: MCP’s own read-only-by-default design; independent guidance on constrained, surgical agent queries; a named industry pattern for “a query that succeeds but lies”; a recommendation that agents stay advisory, not authorized to change state. This is a known class, not an anecdote.

[21:00–23:00] The counterfactual, closing the loop. Play out what the false escalation in replay 3 would have cost a team that skipped the context layer — full confidence, fast, wrong. Return to the locum-doctor framing from the chapter: the problem was never the agent’s knowledge, it’s the notes only the regular doctor would think to leave.

[23:00–25:00] Takeaways and close. Four things attendees can do Monday: (1) replay your own resolved incidents before trusting any agent’s triage claim; (2) build a small, on-demand context layer, not a giant upfront one; (3) enforce read-only at the token’s permission scope, never at the prompt; (4) keep an automated declared-config checker running alongside the agent — it catches a different class of failure entirely. Close on the title line.

5. Notes for reviewers / “is this a case study” field

Yes — production case study, not a demo or vendor pitch. All four incidents replayed are real, already-resolved production incidents; company name, internal domains and resource IDs are generalized, consistent with how the source material (a full-length book on this production observability implementation) already handles this. Not previously presented at any Linux Foundation event.

6. Relevant CNCF / ecosystem projects

- OpenTelemetry (the telemetry the agent queries)
- Prometheus (the metrics query surface the MCP tool exposes)
- Model Context Protocol (the connector standard the talk is about)
- Grafana (Loki/Tempo/Mimir), noted as platform context if the form allows non-CNCF mentions

7. Speaker bio (draft)

Predrag Mujkovic is a Senior DevOps/SRE engineer with close to a decade of experience across cloud and on-premises infrastructure, currently running observability for a weather-data analytics platform on AWS (ECS Fargate, Aurora/RDS, Terraform) with OpenTelemetry and the Grafana LGTM stack (Loki, Grafana, Tempo, Mimir). He is the author of a full-length book documenting that platform's observability implementation from the ground up, based on the real production system.

8. One-line pitch (social/preview card)

We gave an AI agent MCP access to our telemetry and tested it against four real incidents — here's exactly where it got the diagnosis right, where it confidently got it wrong, and the one architectural decision that kept “confidently wrong” from becoming “silently destructive.”

9. Illustrative diagrams (slide candidates — not for the CFP form itself)

Labels are currently Serbian-only in the book; re-render in English before they go on a slide in front of this audience.

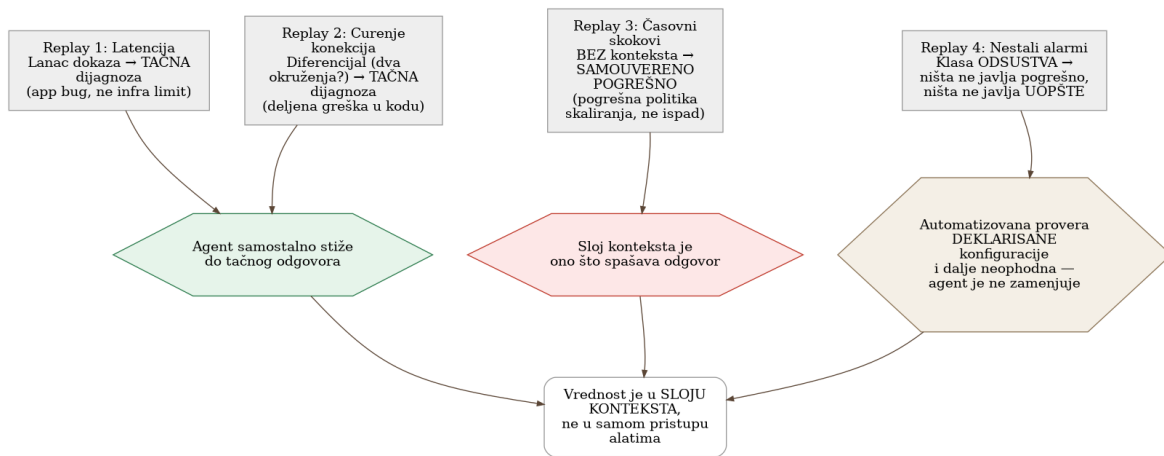


Figure 1: Four replayed incidents: two where the agent independently reaches the correct diagnosis, one where it would confidently get it wrong without the context layer, and one from the “class of absence.”

10. Before you submit — checklist

1. Confirm actual title/abstract character limits on the live Sessionize form and trim §2 accordingly.
2. Confirm whether the form allows targeting more than one co-located event.
3. This hasn't been presented before — the “differs from a prior talk” note doesn't apply, but keep one sentence ready in case the form asks generally.
4. Re-render the two Serbian-labeled diagrams in English before they reach a slide.

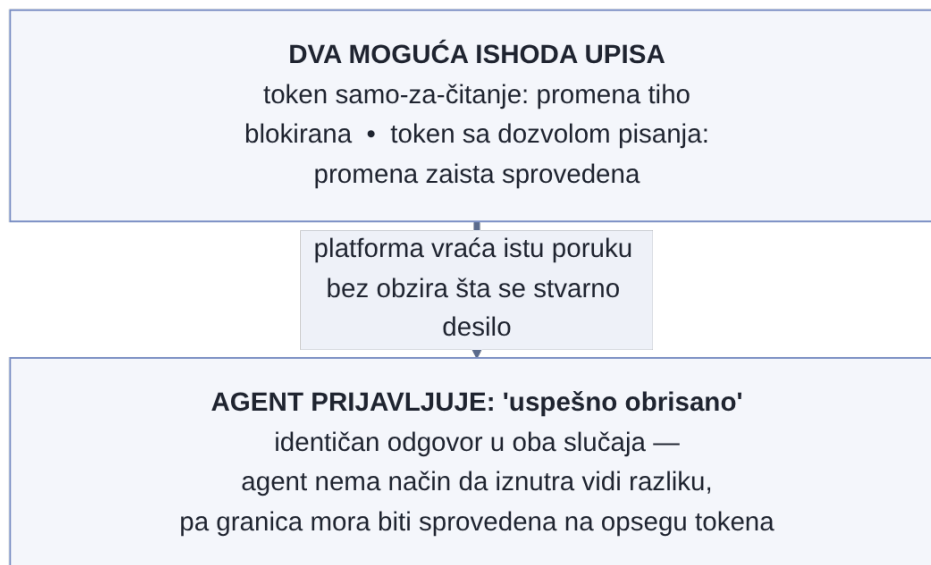


Figure 2: False success: a blocked write and a real write return an identical “success” message.

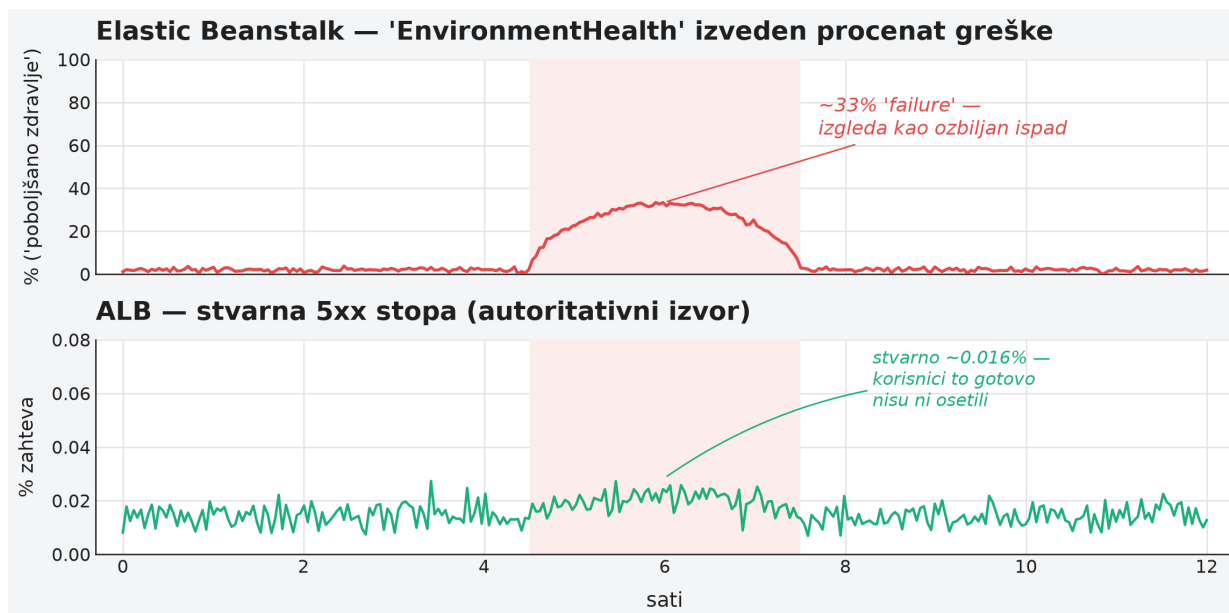


Figure 3: The seductive but wrong signal: derived “environment unhealthiness” vs. the authoritative 5xx rate — the replay-3 dashboard.